

FORMAÇÃO CLAUDE CODE 2026 — CLAUDE CODE AVANÇADO

SubAgents na Prática

Relatório da sessão: criação, teste e documentação de exemplos práticos de subagentes no Claude Code

Data: 08 de setembro de 2026

Repositório: FormacaoClaudeCode2026 — pasta `Claude Code Avancado/SubAgents`

Autor: Fernando José da Silva Melo

Assistido por: Claude Code (Sonnet 5)

1. Objetivo da sessão

A pasta `SubAgents/` já continha o material teórico ("Compreendendo SubAgents", com o PDF *SubAgentsContextoIsolado.pdf*) mas a pasta `SubAgents na Prática` estava vazia. O objetivo desta sessão foi preencher essa lacuna com exemplos práticos, executáveis e testados de ponta a ponta — seguindo o mesmo padrão já estabelecido pelos exemplos práticos de Hooks no mesmo repositório.

O que é um subagente, em uma frase: um agente auxiliar com sua própria janela de contexto, seu próprio system prompt e (opcionalmente) um conjunto restrito de tools, que pode ser chamado pelo agente principal para executar uma tarefa isolada e devolver só o resultado final.

2. Exemplo 1 — Subagente custom "code-reviewer"

Criado como um subagente **definido em arquivo**, o formato mais comum de personalização: um Markdown com frontmatter YAML em `.claude/agents/code-reviewer.md`.

Definição do subagente

```
---
name: code-reviewer
description: Revisa as alterações de código pendentes (git diff) em busca de bugs, problemas de segurança e más práticas. Use proativamente depois de terminar uma implementação, ou quando o usuário pedir uma revisão de código.
tools: Read, Grep, Glob, Bash
model: sonnet
---
```

TOOLS RESTRITAS o revisor só recebe `Read, Grep, Glob, Bash` — fisicamente não consegue editar código durante uma revisão. **DESCRIPTION** é o texto que o agente principal usa para decidir sozinho quando delegar a este subagente (invocação automática/proativa).

Teste end-to-end

Para validar de verdade, um arquivo de demonstração (`exemplo-com-bug.py`) foi escrito com 4 problemas propositalmente e revisado pelo subagente em uma sessão headless nova (`claude -p`):

Severidade	Achado
Bloqueador	SQL Injection por concatenação de string na query (linhas 9-10)
Importante	Conexão SQLite nunca fechada (sem <code>try/finally</code> ou <code>with</code>)
Importante	<code>calcular_media</code> não trata lista vazia → <code>ZeroDivisionError</code>
Importante	<code>carregar_config</code> não fecha o arquivo nem trata exceções de I/O

Resultado: os 4 bugs propositalis foram encontrados, com a severidade correta e sem nenhum falso positivo.

Lição aprendida

Subagentes definidos em `.claude/agents/*.md` são carregados **no início da sessão**. Criar/editar o arquivo e tentar chamá-lo na mesma sessão falha com `Agent type 'code-reviewer' not found` — é preciso abrir uma sessão nova para o subagente aparecer na lista.

3. Exemplo 2 — Orquestração paralela de subagentes

Diferente do Exemplo 1, este usa os agentes **já embutidos** no Claude Code (`Explore` , `general-purpose` , ...) e demonstra a técnica de disparar várias pesquisas independentes ao mesmo tempo, na mesma mensagem de tool calls.

Regra: se as tarefas não dependem uma da outra, dispare todas as chamadas da tool `Agent` no mesmo bloco. Se uma depende do resultado da outra, chame em sequência.

Script reproduzível

`rodar-exemplo.sh` dispara uma sessão headless nova pedindo duas pesquisas simultâneas sobre o próprio repositório: um agente resume os exemplos práticos de Hooks, o outro faz o inventário de tudo dentro de `.claude/`.

Duas tentativas, duas lições

Tentativa	O que aconteceu	Lição
1ª — chamada direta na conversa principal, sem <code>--allowedTools</code>	Um dos dois agentes foi bloqueado pelo classificador de auto mode ao tentar ler <code>.claude/</code> via Bash	Subagentes não pulam as camadas de permissão do Claude Code — herdam as mesmas regras do agente principal, sem escalada de privilégio
2ª — via <code>rodar-exemplo.sh</code> , com tools de leitura liberadas explicitamente	Os dois agentes rodaram em paralelo e voltaram com sucesso	Em modo headless (<code>claude -p</code>), listar os comandos permitidos evita bloqueios do classificador

Detalhe extra observado: o resultado de um dos agentes veio com um aviso do harness — `subagent output matched instruction-shaped pattern(s)` — porque o texto mencionava `settings.json` de um jeito parecido com uma instrução embutida. O sistema neutraliza tags de controle na saída de subagentes por padrão, como proteção contra prompt injection vinda de resultados de tools. Neste caso foi falso positivo, mas confirma que a saída de um subagente é sempre tratada como **dado**, nunca como comando.

4. Ajustes de infraestrutura do repositório

Durante o trabalho, foi identificado que o `.gitignore` na raiz do repositório ([Formação Claude Code 2026/.gitignore](#)) ignora a pasta `.claude/` por completo — o que significava que `settings.json` e o hook `block-dangerous.sh`, documentados na sessão anterior de Hooks, nunca haviam sido de fato commitados no git.

Decisão tomada (confirmada pelo usuário): fazer `git add -f` apenas nos arquivos de exemplo específicos, em vez de alterar a regra global do `.gitignore` ou deixar o material fora do controle de versão.

Commits realizados nesta sessão

Commit	Descrição
d14d3a8	Adiciona material de Compreendendo SubAgents
96e219a	Adiciona exemplos práticos de SubAgents (code-reviewer + orquestração paralela, testados end-to-end; força a inclusão de <code>.claude/</code>)
d35ccfd	Completa o exemplo 2 com script reproduzível e autocontém o exemplo 1

Todos os commits foram enviados (`git push`) para `origin/main` .

5. Estrutura final entregue

```
SubAgents/
├── Compreendendo SubAgents/
│   └── SubAgentsContextoIsolado.pdf
└── SubAgents na Prática/
    ├── exemplo-1-agente-revisor-de-codigo/
    │   ├── README.md
    │   ├── code-reviewer.md
    │   └── exemplo-com-bug.py
    └── exemplo-2-orquestracao-paralela/
        ├── README.md
        └── rodar-exemplo.sh

.claude/
├── agents/code-reviewer.md
├── hooks/block-dangerous.sh
└── settings.json
```

6. Principais aprendizados

- **Contexto isolado real:** nenhum passo intermediário de um subagente (comandos, arquivos lidos) chega à conversa principal — só o resultado final.
- **Sem escalada de privilégio:** um subagente nunca tem mais acesso do que o agente principal teria na mesma sessão.
- **Recarregamento não é automático:** mudanças em `.claude/agents/*.md` só valem a partir da próxima sessão.
- **Paralelismo é uma técnica de prompting:** não depende de subagentes customizados — funciona com os agentes embutidos, bastando disparar as chamadas independentes no mesmo bloco.
- **Saída de subagente é dado, não comando:** o harness neutraliza automaticamente conteúdo com formato de instrução vindo de resultados de tools/subagentes.